

Análise de Código Java

ManualSemaphore — Sincronização Manual de Threads



Código Analisado

```
public class ManualSemaphore {
    private int permits;

    public ManualSemaphore(int permits) { this.permits = permits; }

    public synchronized void acquire() throws InterruptedException {
        while (permits == 0) wait();
        permits--;
    }

    public synchronized void release() {
        permits++;
        notify();
    }

    public static void main(String[] args) {
        ManualSemaphore sem = new ManualSemaphore(2);
        Runnable task = () -> {
            try {
                sem.acquire();
                System.out.println(Thread.currentThread().getName() + " acquired");
                Thread.sleep(200);
                sem.release();
                System.out.println(Thread.currentThread().getName() + " released");
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        };
        for (int i = 0; i < 4; i++) new Thread(task, "T" + i).start();
    }
}
```

Objetivo Geral

A classe **ManualSemaphore** implementa manualmente um semáforo de contagem em Java, controlando o acesso concorrente a um recurso compartilhado. O objetivo é limitar o número de threads que podem executar simultaneamente uma secção crítica, neste caso permitindo no máximo 2 threads ativas ao mesmo tempo entre 4 que competem pelo acesso.

Lógica e Algoritmo

O semáforo mantém um contador interno (*permits*) inicializado com o número de permissões disponíveis. Em **acquire()**, se não existirem permissões, a thread entra em espera (*wait()*) libertando o monitor; quando outra thread chama **release()**, incrementa o contador e acorda uma thread em espera com *notify()*. O ciclo *while* em *acquire()* protege contra *spurious wakeups*, sendo uma prática indispensável na programação concorrente em Java.

Conceitos Java Envolvidos

O código utiliza **sincronização intrínseca** (*synchronized*), o mecanismo de **wait/notify** do monitor de objecto, **expressões lambda** para definir a tarefa concorrente, e a API de **threads** com *Thread.sleep()* e *Thread.currentThread()*. A classe simula o comportamento de *java.util.concurrent.Semaphore* sem recorrer às classes de alto nível do pacote *java.util.concurrent*.

Complexidade

A complexidade temporal de *acquire()* e *release()* é **O(1)** em condições normais, excluindo o tempo de espera por permissões disponíveis. O espaço ocupado é **O(1)**, pois o estado do semáforo resume-se a um único inteiro. O custo real em cenários de alta contention está no bloqueio/desbloqueio de threads, gerido internamente pela JVM.

Decisões de Design e Limitações

A utilização de *notify()* em vez de *notifyAll()* pode causar **starvation** se múltiplas threads estiverem à espera, pois apenas uma é acordada por chamada. Além disso, a implementação não é *fair* (não garante ordem FIFO), ao contrário de *Semaphore(n, true)* da JDK. Para produção, recomenda-se usar *java.util.concurrent.Semaphore*, que oferece suporte a fairness, timeouts e interrupção controlada.

